



CURSO BÁSICO DE PROGRAMACIÓN EN JAVA

CONSTRUCTORES Y ENCAPSULAMIENTO (POO)

Tutor/es: Brandon Jiménez, Christopher Benavides,
Jhon Laverde, Dayana Jácome

1



INDICE

01. Introducción

02. ¿Que es un constructor?
(concepto + finalidad)

03. Sintaxis de un constructor

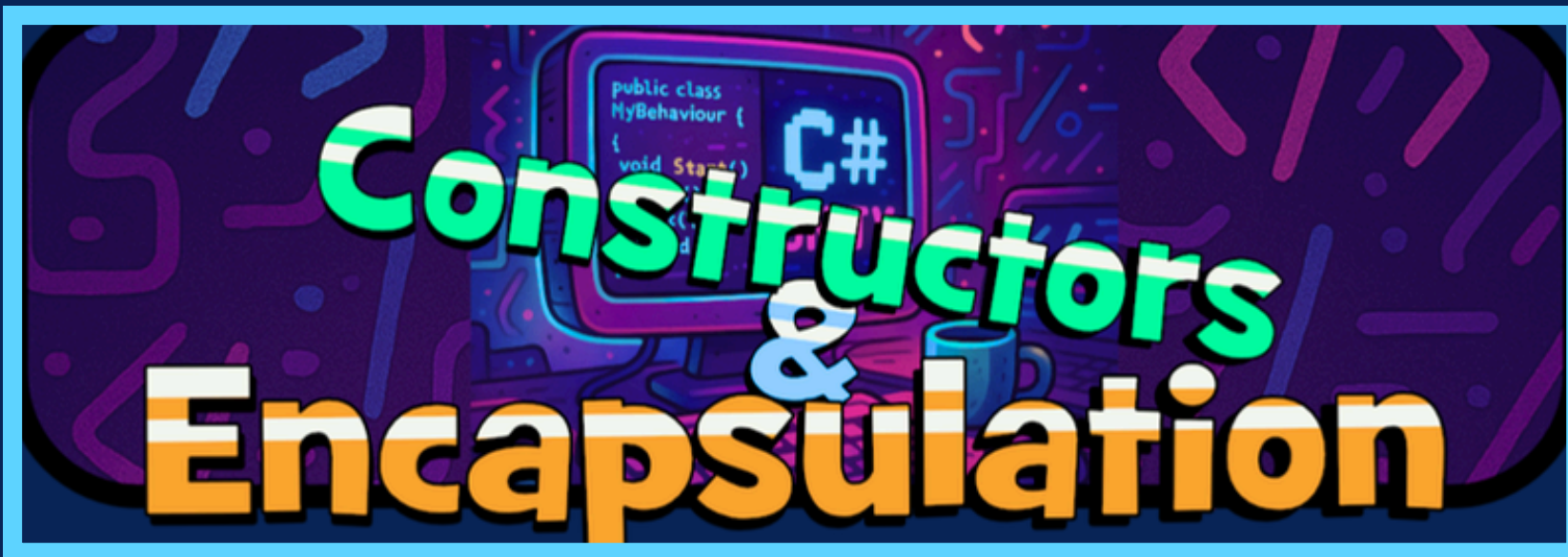
04. Constructor por defecto y parametrizado

05. Sobrecarga de constructores

06. Encapsulamiento

07. Modificadores de acceso

08. Métodos getters y setters



2



INTRODUCCIÓN

Cuando trabajamos con clases, no basta solo con definirlas, sino que necesitamos una forma de crear objetos correctamente y asegurarnos de que tengan valores válidos desde el inicio. **Para eso existen los constructores.**

Además, también es importante proteger la información de los objetos, evitando que cualquier parte del programa la modifique sin control. Esto se logra mediante el **encapsulamiento** y el uso de herramientas como los **modificadores de acceso y los métodos getters y setters.**



¿QUÉ ES UN CONSTRUCTOR?

Un constructor es un método especial que sirve para crear e inicializar un objeto usando **new**

```
public CuentaBancaria(String titular, String numeroCuenta, double saldoInicial) {
```

```
//CREAR EL OBJETO ---> constructor
```

```
CuentaBancaria cuenta1 = new CuentaBancaria(nombre, cuenta, saldoInicial);
```

SINTAXIS DE UN CONSTRUCTOR

Características importantes:

- Tiene el mismo nombre de la clase
- NO tiene tipo de retorno (ni void)
- Se ejecuta automáticamente



Tenemos una clase Mascota....

La mascota necesita:

- nombre
- edad
- raza

```
Clase Objeto = new Constructor()
```



CONSTRUCTOR POR DEFECTO Y PARAMETRIZADO

Constructor por defecto

- Es un constructor que **NO recibe datos**
- Sirve para crear el objeto con valores iniciales básicos

```
public class Persona {  
    String nombre;  
    int edad;  
  
    // Constructor por defecto  
    public Persona() {  
        nombre = "Sin nombre";  
        edad = 0;  
    }  
}
```

Constructor parametrizado

- Es un constructor que **recibe datos al crear el objeto**
- Tú decides los valores

```
public class Persona {  
    String nombre;  
    int edad;  
  
    public Persona(String nombre, int edad) {  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
}
```

```
Persona p2 = new Persona("Ana", 20);
```

5

SOBRECARGA DE CONSTRUCTORES



Tener VARIOS constructores en la misma clase

Pero:

deben ser diferentes ---> (parámetros distintos)

```
public class Persona {  
  
    String nombre;  
    int edad;  
  
    // 1. Constructor por defecto  
    public Persona() {  
        nombre = "Desconocido";  
        edad = 0;  
    }  
}
```

```
// 2. Solo nombre  
public Persona(String nombre) {  
    this.nombre = nombre;  
    edad = 0;  
}  
  
// 3. Nombre y edad  
public Persona(String nombre, int edad) {  
    this.nombre = nombre;  
    this.edad = edad;  
}
```



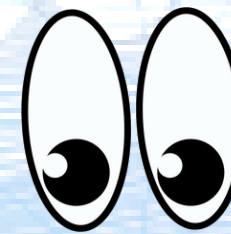
ENCAPSULAMIENTO

- Es proteger los datos del objeto
- No dejar que cualquiera cambie los datos directamente

```
private String titular;  
private String numeroCuenta;  
private double saldo; // public double saldo X
```

- Un ejemplo común de encapsulación en Java es tener atributos privados y proporcionar métodos públicos de acceso, conocidos como **"getters"** y **"setters"**, para leer y modificar esos atributos de forma controlada.

Para lograr la encapsulación en Java, se utilizan **modificadores de acceso**



7

MODIFICADORES DE ACCESO

Los modificadores de acceso permiten dar un nivel de seguridad mayor a nuestras aplicaciones restringiendo el acceso a diferentes atributos, métodos, constructores asegurándonos que el usuario deba seguir una "ruta" especificada por nosotros para acceder a la información.

Visibilidad	Public	Private	Protected	Default*
Desde la misma clase	✓	✓	✓	✓
Desde una subclase	✓	✓	✓	✗
Desde otra clase (no subclase)	✓	✗	✗	✗

**Con default, me refiero a omitir el modificador de acceso.*



GETTERS Y SETTERS

Son métodos para acceder y modificar datos privados

Método get

```
public class Persona {  
    private String nombre;  
  
    public String getNombre() {  
        return nombre;  
    }  
}
```

Método set

```
public class Persona {  
    private String nombre;  
  
    public void setNombre(String nuevoNombre) {  
        nombre = nuevoNombre;  
    }  
}
```

EJEMPLO

10



CONCLUSIONES

- 1** . Existen diferentes formas de inicializar objetos
Usando constructores por defecto y parametrizados
- 2** El encapsulamiento permite controlar el acceso a los datos, evitando modificaciones indebidas
- 3** La sobrecarga de constructores permite crear objetos de distintas maneras dentro de una misma clase, adaptándose a diferentes necesidades.
- 4** Palabras como public, private y protected permiten regular quién puede acceder a los datos y métodos



TAREA

Desarrollar una clase llamada Mascota que permita representar una mascota con sus datos y determinar su estado según su edad.

1. Atributos (Encapsulamiento)

La clase debe tener los siguientes atributos privados:

- nombre (String)
- edad (int)
- tipo (String)

12



TAREA

2. Constructores

- **Constructor por defecto**

Debe inicializar:

- nombre = "Sin nombre"
- edad = 0
- tipo = "Desconocido"

- **Constructor con un parámetro**

- Recibe: nombre
- Inicializa:
 - edad = 0
 - tipo = "Desconocido"

- **Constructor completo**

- Recibe: nombre, edad y tipo
- Debe usar this para asignar valores

13

EJEMPLO

```
// Setters con validación
public void setEdad(int edad) {
    if (edad >= 0) {
        this.edad = edad;
    } else {
        System.out.println("Edad no válida");
    }
}
```

```
Mascota m1 = new Mascota();
Mascota m2 = new Mascota("Luna");
Mascota m3 = new Mascota("Max", 5, "Perro");
```

```
// Modificar datos
m1.setEdad(3);
m1.setTipo("Gato");
```

```
<terminated> Main2 [Java Application] C:\Users\USER
----- MASCOTA 1 -----
Nombre: Sin nombre
Edad: 1
Tipo: Gato
Estado: Joven

----- MASCOTA 2 -----
Nombre: Luna
Edad: 2
Tipo: Conejo
Estado: Joven

----- MASCOTA 3 -----
Nombre: Max
Edad: 5
Tipo: Perro
Estado: Mayor
```



GRACIAS



MIENTRAS
UNOS SUFREN

•POR AMOR•

YO SUFRO
PORQUE
NO COMPILA